

Android 中对象的序列化与Parcelable 介绍

在 Android 中传递对象，需要让对象 “可序列化”，常见的两种方式是 `Serializable` 和 `Parcelable`。咱们用最直白的方式，结合代码讲清楚～

先明确：为什么对象不能直接传？

普通对象是内存中的数据，Intent 传递数据时需要把数据 “打包” 成系统能识别的格式（类似快递需要打包才能运输）。`Serializable` 和 `Parcelable` 就是两种 “打包规则”。

方式 1：Serializable（最简单，适合小白）

步骤 1：让对象类实现 Serializable 接口

比如我们有个 `Person` 类，想传递它的对象：

```
import java.io.Serializable;
// 加上 implements Serializable，表示这个类的对象可以被序列化（打包）
public class Person implements Serializable {
    // 成员变量（要传递的信息）
    public String name;
    public int age;
    // 构造方法（方便创建对象）
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

- 注意：`Serializable` 是个空接口，只做 “标记” 用，告诉系统这个类可以被打包。

步骤 2：在发送方传递对象

和传递字符串类似，用 `Bundle` 或 `Intent` 直接放进去：

```
// 1. 创建要传递的对象
Person person = new Person("小明", 18);
// 2. 放入 Intent（和传字符串几乎一样）
Intent intent = new Intent(this, ActRecordActivity.class);
// 直接用 putExtra 放对象，键名自己定义（比如"person_data"）
intent.putExtra("person_data", person);
// 3. 启动 Activity
startActivity(intent);
```

步骤 3：在接收方获取对象

在 `ActRecordActivity` 中，用 `getSerializableExtra` 取出：

```
// 在 onCreate 方法中获取
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_act_record);
    // 1. 获取 Intent
    Intent intent = getIntent();
    // 2. 取出对象（注意强转成 Person 类型）
    Person person = (Person) intent.getSerializableExtra("person_data");
    // 3. 使用对象的数据
    if (person != null) {
        Log.d("接收数据", "名字：" + person.name + "，年龄：" + person.age);
    }
}
```

方式 2：Parcelable（效率高，Android 推荐，稍复杂）

如果是频繁传递对象（比如列表中的数据），用 `Parcelable` 更高效。步骤稍多，但套路固定。

步骤 1：让对象类实现 Parcelable 接口

还是 `Person` 类，这次按规则实现接口：

```

import android.os.Parcel;
import android.os.Parcelable;
public class Person implements Parcelable {
    public String name;
    public int age;
    // 构造方法
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    // 下面是固定写法，按模板改就行
    // 从 Parcel 中读数据（解包）
    protected Person(Parcel in) {
        name = in.readString(); // 按顺序读，和 write 对应
        age = in.readInt();
    }
    // 必须有的 Creator，用于创建对象
    public static final Creator<Person> CREATOR = new Creator<Person>() {
        @Override
        public Person createFromParcel(Parcel in) {
            return new Person(in); // 调用上面的构造方法
        }
        @Override
        public Person[] newArray(int size) {
            return new Person[size];
        }
    };
    // 不用管，返回 0 就行
    @Override
    public int describeContents() {
        return 0;
    }
    // 把数据写入 Parcel（打包）
    @Override
    public void writeToParcel(Parcel dest, int flags) {
        dest.writeString(name); // 按顺序写
        dest.writeInt(age);
    }
}

```

- 重点：writeToParcel（打包时写数据）和 Person(Parcel in）（解包时读数据）的顺序必须一致！

步骤 2：发送方传递对象

和 `Serializable` 类似，只是用 `putExtra` 直接放：

```
// 创建对象
Person person = new Person("小红", 20);
// 放入 Intent
Intent intent = new Intent(this, ActRecordActivity.class);
intent.putExtra("person_data", person); // 键名还是自己定义
// 启动
startActivity(intent);
```

步骤 3：接收方获取对象

用 `getParcelableExtra` 取出：

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_act_record);
    Intent intent = getIntent();
    // 注意用 getParcelableExtra，同样强转
    Person person = intent.getParcelableExtra("person_data");
    if (person != null) {
        Log.d("接收数据", "名字: " + person.name + ", 年龄: " + person.age);
    }
}
```

小白总结

方式	特点	用法难度	推荐场景
Serializable	代码简单，直接实现接口	简单	一般场景，偶尔传对象
Parcelable	效率高，Android 推荐	稍复杂	频繁传对象（如列表）

新手首选 Serializable，代码少不容易错，基本能满足大部分需求～ 传递时记住：对象类先“打标记”（实现接口），然后像传字符串一样用 `putExtra` 放进去，接收时用对应的方法取出来就行。

（注：文档部分内容可能由 AI 生成）